

claude-windows / c47f0851-d75f-45b6-9126-e48544381426

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c47f0851-d75f-45b6-9126-e48544381426\subagents\workflows\wf_5af9a132-4ce\agent-a61f0a3c82f86e433.jsonl`
- SHA-256: `f3fae3c161580f46e8d2b641c8af40a3af274783467f542aca42e1b65caf3028`
- Source modified: `2026-06-20T09:05:41+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_5af9a132-4ce`
- session_id: `c47f0851-d75f-45b6-9126-e48544381426`
```

Transcript

1. user (2026-06-20T09:03:01.581Z)

Adversarially verify this review finding against the ACTUAL code in the working tree. Default to 'false-positive' unless you can point to the concrete line(s) and a real failure scenario. Context: Repo: badminton-highlight-indexer (Python). Branch feat/native-wasb-runner off origin/master.

The change builds the M3.5 Linux-native WASB detector runner + wires real extraction into the worker.

Files changed/added (read them in the WORKING TREE ? changes are uncommitted):

```
- backend/pipeline/detectors/native_wasb_runner.py (NEW ? the native runner)
- backend/pipeline/detectors/wasb_infer.py (added --device, threaded device into
_build_cfg/run/run_video_streaming/main; device-keyed the resumable manifest)
- backend/pipeline/segmenters/trajectory_hybrid.py (_resolve_runner: new 'native' branch +
_build_native_runner base hook; generalized 'WSL env' log lines)
- backend/pipeline/segmenters/wasb_hybrid.py (_build_native_runner override)
- backend/pipeline/segmenters/fusion_hybrid.py (_build_native_runner override; WASB
substrate only)
- backend/config/models.py (WasbIndexConfig.device + .python_bin;
detector_impl doc mentions 'native')
- backend/worker/__main__.py (cmd_train: --trajectory-source
{synth,detector} + --segmenter; _resolve_train_detector; detector path pulls videos + runs
runner)
- tests/test_native_wasb_runner.py, tests/test_worker.py (new tests)
```

DESIGN INTENT / CONTRACTS (judge against these):

```
- The EXISTING WSL runner is backend/pipeline/detectors/wasb_runner.py (WasbRunner,
WslCommandMixin). The native runner must be a sibling that runs the SAME wasb_infer.py natively
(no wsl, no 'conda activate' ? it invokes the wasb conda env python BY PATH, no /mnt path
translation) and emit a BYTE-IDENTICAL Frame,Visibility,X,Y CSV with the SAME filename
'{stem}_{vid12}_wasb.csv'.
- ACCEPTANCE GATE = byte-parity: same clip via WSL (Windows) vs native (Linux, device=cuda) ->
identical CSV + identical candidate windows. device default 'cuda' must keep wasb_infer's Hydra
overrides identical to before (parity).
- Selected via the detector_impl seam: indexing.detector_impl / RALLY_DETECTOR_IMPL = 'native'.
- Constraint: 'backend' must NOT import 'training' (worker shells out). The default trajectory
source stays 'synth' so the CPU e2e is unchanged.
- The code RUNS on Linux (GPU box) but the TESTS run on the dev's Windows machine; cross-
platform path handling matters.
- weights/repo/python/scratch come from env first
(WASB_WEIGHTS/WASB_REPO_DIR/WASB_PYTHON/WASB_SCRATCH) then indexing.wasb.* then default.
```

FINDING [seam-config] (major) @ backend/worker/__main__.py:183 (_resolve_train_detector):
_resolve_train_detector calls seg._resolve_runner(config.get('indexing', {})) with NO
try/except. With detector_impl='native' and a non-WASB detector (--segmenter tracknet_hybrid, or
fusion_hybrid with indexing.fusion.substrate='tracknet'), _build_native_runner raises
NotImplementedError, which propagates uncaught out of cmd_train and crashes the worker with a

raw traceback ? instead of the function's established graceful pattern (print a clear message + return None, which cmd_train turns into `return 2`). Every other failure in this function (unknown segmenter, non-trajectory segmenter, failed healthcheck) is handled this way; the native-refusal path is the one gap. cmd_infer -> process_video -> _resolve_runner has the same exposure (no except clause), so a `worker infer --segmenter tracknet\_hybrid` with RALLY_DETECTOR_IMPL=native also tracebacks.

WHY: Reproduced: with RALLY_DETECTOR_IMPL=native, _resolve_train_detector(args(segmenter='tracknet_hybrid'), config) raises an UNCAUGHT NotImplementedError ('detector_impl=native is not supported for the TrackNet family...'). A user running `python -m backend.worker train --trajectory-source detector --segmenter tracknet\_hybrid` (or fusion with a tracknet substrate) on the GPU box with the native impl env set gets a Python crash + traceback rather than a one-line actionable error and a clean non-zero exit code.

PROPOSED FIX: Wrap the _resolve_runner call in _resolve_train_detector in try/except NotImplementedError: print(f"[worker] {e}") and return None ? mirroring the other early-return error branches. (Optionally do the same around the segmenter call in cmd_infer so a native+non-WASB misconfig there also fails cleanly with rc!=0 instead of a traceback.)

2. assistant (2026-06-20T09:03:03.930Z)

I'll verify this finding against the actual code in the working tree. Let me start by reading the relevant files.

3. user (2026-06-20T09:03:04.506Z)

```
1      """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3      python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4          [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5      python -m backend.worker train ...          # Gen-0 train-from-bucket (follow-up)
6
7      The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
8      ``local://``
9      is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing
10     local runs
11     are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
12     ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
13     key) ?
14     the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
15
16     This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
17     CLI uses
18     (validator registry ? segmenter ? report), never a forked pipeline.
19     """
20     from __future__ import annotations
21
22     import argparse
23     import csv
24     import json
25     import os
26     import sys
27     import tempfile
28     from typing import Any, List
29
30     from backend import storage
31     from backend.config import load_config
32     from backend.infrastructure.database import Database
33     from backend.pipeline.segmenters.base import segmenter_registry
34     from backend.pipeline.validators.base import registry as validator_registry
35     from backend.reporting import emit_indexer_report
36     from backend.results import Failure
37     from backend.utils.hashing import compute_video_id
38
39     def _coerce(v: str) -> Any:
```

```

37         """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
str."""
38         low = v.lower()
39         if low in ("true", "false"):
40             return low == "true"
41         for cast in (int, float):
42             try:
43                 return cast(v)
44             except ValueError:
45                 pass
46         return v
47
48
49     def _apply_overrides(config: Any, sets: List[str]) -> None:
50         """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
indexing.skip_ai_handoff=true)."""
51         for kv in sets or []:
52             key, sep, val = kv.partition("=")
53             if not sep:
54                 raise ValueError(f"--set expects k=v, got {kv!r}")
55             parts = key.split(".")
56             obj = config
57             for p in parts[:-1]:
58                 obj = getattr(obj, p)
59             setattr(obj, parts[-1], _coerce(val))
60
61
62     def _write_intervals_csv(segments: List[dict], path: str) -> None:
63         """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
friendly
64         artifact (same schema as the rally-annotator labels)."""
65         with open(path, "w", newline="") as f:
66             w = csv.writer(f)
67             w.writerow(["start", "end", "shot_count", "ending_reason"])
68             for s in segments:
69                 w.writerow([
70                     f'{float(s.get("start_time", 0.0)):.3f}',
71                     f'{float(s.get("end_time", 0.0)):.3f}',
72                     s.get("shot_count", ""),
73                     s.get("ending_reason", s.get("shot_count_confidence", "")),
74                 ])
75
76
77     def _write_report_json(video_id: str, segments: List[dict], status: str, path: str) ->
None:
78         with open(path, "w") as f:
79             json.dump({
80                 "video_id": video_id,
81                 "status": status,
82                 "segment_count": len(segments),
83                 "segments": [{"start": float(s.get("start_time", 0.0)),
84                             "end": float(s.get("end_time", 0.0))} for s in segments],
85             }, f, indent=2)
86
87
88     def cmd_infer(args: argparse.Namespace) -> int:
89         config = load_config(args.config) if args.config else load_config()
90         _apply_overrides(config, args.set)
91         storage_cfg = config.storage.model_dump()
92
93         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
94         os.makedirs(scratch, exist_ok=True)
95         config.output.output_dir = scratch
96
97         # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download

```

```

to scratch.
98     local_video = storage.fetch(args.video_uri, os.path.join(scratch, "in.mp4"),
config=storage_cfg)
99     print(f"[worker] pulled {args.video_uri} -> {local_video}")
100
101     # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
102     video_id = compute_video_id(local_video)
103
104     # 3. VALIDATE (same registry as the main CLI).
105     val_results, val_context = validator_registry.run_all_with_context(local_video,
config)
106     failures = [r for r in val_results if not r.passed]
107     db = Database(os.path.join(scratch, config.output.db_name))
108     db.add_video(video_id, local_video, "failed" if failures else "processed",
109                 [r.model_dump() for r in val_results])
110
111     segments: List[dict] = []
112     segmenter_actual = None
113     segmentation_ran = False
114     status = "failed"
115     try:
116         if failures:
117             print("[worker] validation FAILED: "
118                   + "; ".join(f"{r.validator_name}: {r.message}" for r in failures))
119             return 2
120         seg_name = args.segmenter or config.indexing.default_segmenter
121         segmenter_cls = segmenter_registry.get(seg_name)
122         if not segmenter_cls:
123             print(f"[worker] unknown segmenter: {seg_name!r} "
124                   f"(have {list(segmenter_registry.list_available())})")
125             return 2
126         segmenter_actual = seg_name
127         result = segmenter_cls(db, config).process_video(video_id, local_video,
max_frames=args.max_frames)
128         segmentation_ran = True
129         if isinstance(result, Failure):
130             print(f"[worker] segmenter FAILED: {result.message}")
131             return 3
132         segments = result.value if hasattr(result, "value") else result
133         status = "success"
134     finally:
135         # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
136         emit_indexer_report(
137             db=db, video_id=video_id, video_path=local_video, config=config,
138             val_results=val_results, val_context=val_context,
139             segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
140             was_reingest=False, segmentation_ran=segmentation_ran,
141         )
142
143     # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
144     out_local = os.path.join(scratch, "outputs", video_id)
145     os.makedirs(out_local, exist_ok=True)
146     _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
147     _write_report_json(video_id, segments, status, os.path.join(out_local,
"report.json"))
148
149     out_prefix = args.out_uri.rstrip("/")
150     pushed = []
151     for fn in sorted(os.listdir(out_local)):
152         uri = f"{out_prefix}/outputs/{video_id}/{fn}"
153         storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
154         pushed.append(uri)
155

```

```

156         print(f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):")
157         for u in pushed:
158             print("    ", u)
159         return 0
160
161
162     def _resolve_train_detector(args: argparse.Namespace, config: Any):
163         """Build the trajectory DetectorRunner for `--trajectory-source detector`, honouring
164         detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU
box,
165         stub=CPU mock). Returns the runner, or prints an error + returns None.
166
167         Reuses the segmenter's `_resolve_runner` seam so the worker and the live CLI build
the
168         detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo)
has
169         no shuttle detector and is rejected.
170         """
171         from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
172
173         seg_cls = segmenter_registry.get(args.segmenter)
174         if not seg_cls:
175             print(f"[worker] unknown segmenter: {args.segmenter!r} "
176                   f"(have {list(segmenter_registry.list_available())})")
177             return None
178         seg = seg_cls(None, config)
179         if not isinstance(seg, TrajectoryHybridSegmenter):
180             print(f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
181                   f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid /
fusion_hybrid).")
182             return None
183         runner, _ = seg._resolve_runner(config.get("indexing", {}))
184         ok, msg = runner.healthcheck()
185         if not ok:
186             print(f"[worker] detector environment not ready: {msg}")
187             return None
188         print(f"[worker] detector ready ({args.segmenter}): {msg}")
189         return runner
190
191
192     def cmd_train(args: argparse.Namespace) -> int:
193         """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest ->
shell out
194         to training.gen0.harness (backend must NOT import training) -> push the artifact
bundle.
195
196         Two trajectory sources (the train pipeline + harness are identical either way):
197         * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent
synthetic
198             shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box /
CI.
199         * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test).
200
This
201         is the M3.5 "first real training" path; only the trajectory source flips.
202         """
203         import subprocess
204
205         config = load_config(args.config) if args.config else load_config()
206         _apply_overrides(config, args.set)
207         storage_cfg = config.storage.model_dump()
208
209         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
210         labels_dir = os.path.join(scratch, "labels")

```

```

211     traj_dir = os.path.join(scratch, "trajectories")
212     videos_dir = os.path.join(scratch, "videos")
213     os.makedirs(labels_dir, exist_ok=True)
214     os.makedirs(traj_dir, exist_ok=True)
215
216     corpus = args.corpus_uri.rstrip("/")
217     label_uris = sorted(u for u in storage.list_uris(f"{corpus}/labels/",
config=storage_cfg)
218                         if u.lower().endswith(".csv"))
219     if len(label_uris) < 2:
220         print(f"[worker] need >=2 labeled videos for leave-one-video-out; found
{len(label_uris)} "
221             f"under {corpus}/labels/")
222     return 2
223
224     # Real-detector source: resolve the runner once + index the bucket's videos/ by
stem.
225     runner = None
226     video_by_stem: dict = {}
227     if args.trajectory_source == "detector":
228         os.makedirs(videos_dir, exist_ok=True)
229         runner = _resolve_train_detector(args, config)
230         if runner is None:
231             return 2
232         for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
233             vname = storage.parse_uri(vuri).name
234             video_by_stem[os.path.splitext(vname)[0]] = vuri
235
236     print(f"[worker] trajectory source: {args.trajectory_source}")
237     specs: List[dict] = []
238     for uri in label_uris:
239         fname = storage.parse_uri(uri).name # e.g.
GX020094_proxy.rallies.csv
240         name = fname.replace(".rallies.csv", "").replace(".csv", "")
241         local_label = storage.fetch(uri, os.path.join(labels_dir, fname),
config=storage_cfg)
242         if args.trajectory_source == "detector":
243             vuri = video_by_stem.get(name)
244             if not vuri:
245                 print(f"[worker] no video for {name!r} under {corpus}/videos/ ?
skipping.")
246                 continue
247             local_video = storage.fetch(vuri, os.path.join(videos_dir,
storage.parse_uri(vuri).name),
248                                     config=storage_cfg)
249             video_id = compute_video_id(local_video)
250             # Real fps/width drive the trajectory frame->time mapping (synth fixes them
at 30/1280).
251             from backend.pipeline.segmenters.trajectory_hybrid import
TrajectoryHybridSegmenter
252             fps, frame_width = TrajectoryHybridSegmenter._probe_fps_width(local_video)
253             traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
254             if not traj:
255                 print(f"[worker] detector produced no trajectory for {name!r} ?
skipping.")
256                 continue
257             specs.append({"name": name, "trajectory": traj, "labels": local_label,
258                         "fps": fps, "frame_width": frame_width})
259         else:
260             from backend.pipeline.detectors.stub_runner import
synth_trajectory_from_labels
261             traj = os.path.join(traj_dir, f"{name}_traj.csv")
262             synth_trajectory_from_labels(local_label, traj, fps=args.fps,
frame_width=args.frame_width)
263             specs.append({"name": name, "trajectory": traj, "labels": local_label,

```

```

264         "fps": args.fps, "frame_width": args.frame_width})
265
266     if len(specs) < 2:
267         print(f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}.")
268         return 2
269     manifest_path = os.path.join(scratch, "manifest.json")
270     with open(manifest_path, "w", encoding="utf-8") as f:
271         json.dump(specs, f, indent=2)
272     src_label = "real detector" if args.trajectory_source == "detector" else "CPU-mock
stub"
273     print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")
274
275     out_dir = os.path.join(scratch, "artifacts")
276     cmd = [sys.executable, "-m", "training.gen0.harness",
277            "--manifest", manifest_path, "--out", out_dir, "--version", args.version]
278     if args.floor:
279         floor_local = (storage.fetch(args.floor, os.path.join(scratch, "floor.json"),
config=storage_cfg)
280                        if "://" in args.floor else args.floor)
281         cmd += ["--floor", floor_local]
282     print("[worker] training:", " ".join(cmd))
283     rc = subprocess.run(cmd).returncode
284     if rc != 0:
285         print(f"[worker] gen0 harness failed (rc={rc})")
286         return rc
287
288     # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
289     bundle = os.path.join(out_dir, args.version)
290     out_prefix = args.out_uri.rstrip("/")
291     pushed = []
292     for fn in sorted(os.listdir(bundle)):
293         uri = f"{out_prefix}/weights/{args.version}/{fn}"
294         storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
295         pushed.append(uri)
296     print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
297     for u in pushed:
298         print("    ", u)
299     return 0
300
301
302 def build_parser() -> argparse.ArgumentParser:
303     p = argparse.ArgumentParser(prog="python -m backend.worker",
304                                description="Storage-aware worker: pull ? run pipeline ?
push.")
305     sub = p.add_subparsers(dest="cmd", required=True)
306
307     pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
308     pi.add_argument("--video-uri", required=True, help="local path / local:// / s3:// /
gcs://")
309     pi.add_argument("--out-uri", required=True, help="output prefix
(outputs/<video_id>/? is appended)")
310     pi.add_argument("--segmenter", default=None, help="default:
indexing.default_segmenter")
311     pi.add_argument("--config", default=None, help="config.json path (default:
./config.json)")
312     pi.add_argument("--scratch", default=None, help="local scratch dir (default: a temp
dir)")
313     pi.add_argument("--max-frames", type=int, default=None)
314     pi.add_argument("--set", action="append", default=[], metavar="k=v",
315                     help="config override, e.g. --set indexing.skip_ai_handoff=true")
316     pi.set_defaults(func=cmd_infer)
317
318     pt = sub.add_parser("train", help="Gen-0 train-from-bucket (labels [+videos] ->
trajectories -> harness)")

```

```

319         pt.add_argument("--corpus-uri", required=True, help="prefix containing labels/ (+
videos/ for "
320             "--trajectory-source detector), e.g. s3://bucket or gcs://bucket")
321         pt.add_argument("--out-uri", required=True, help="output prefix (weights/<version>/
is appended)")
322         pt.add_argument("--version", required=True, help="semver for the artifact bundle")
323         pt.add_argument("--trajectory-source", choices=["synth", "detector"],
default="synth",
324             help="synth = CPU-mock label-consistent trajectories (default, no
GPU); "
325             "detector = REAL shuttle trajectories via the resolved
DetectorRunner "
326             "(needs videos/ in the corpus; honours detector_impl).")
327         pt.add_argument("--segmenter", default="wasb_hybrid",
328             help="trajectory hybrid whose detector is used for --trajectory-
source detector")
329         pt.add_argument("--floor", default=None, help="floor baseline JSON (local path or
storage URI)")
330         pt.add_argument("--config", default=None)
331         pt.add_argument("--scratch", default=None)
332         pt.add_argument("--fps", type=float, default=30.0, help="synth trajectory fps
(detector probes the video)")
333         pt.add_argument("--frame-width", type=float, default=1280.0,
334             help="synth trajectory frame width (detector probes the video)")
335         pt.add_argument("--set", action="append", default=[], metavar="k=v")
336         pt.set_defaults(func=cmd_train)
337         return p
338
339
340     def main(argv=None) -> int:
341         args = build_parser().parse_args(argv)
342         return args.func(args)
343
344
345     if __name__ == "__main__":
346         sys.exit(main())
347

```

4. assistant (2026-06-20T09:03:07.074Z)

Now let me examine the segmenter files to understand the `\_build\_native\_runner` and `\_resolve\_runner` behavior.

5. user (2026-06-20T09:03:07.801Z)

```

1     """Shared engine for trajectory-substrate hybrid segmenters.
2
3     tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a
4     standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
5     now the single home for the whole pipeline shape:
6
7     healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
8     -> P3 AI-provider handoff over padded clips -> P4 DB population/linking
9
10    A concrete subclass supplies only its identity and runner wiring:
11    - ``family`` ? config section under ``indexing.*``, output subdir, and
12      the ``<family>-hybrid-<model>`` detector tag in metadata.
13    - ``display_label`` ? human label for log lines ("WASB", "TrackNet").
14    - ``name`` / ``description`` properties (the registry contract).
15    - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
16      detection_params extras such as weights_path).
17
18    This base is deliberately NOT registered; only concrete subclasses register.
19    """
20    import os

```



```

21     import time
22     import logging
23     import concurrent.futures
24     from typing import Any, Dict, List, Optional, Tuple
25
26     import cv2
27
28     from backend.pipeline.segmenters.base import BasePlaySegmenter
29     from backend.utils.video import get_video_duration, extract_video_chunk
30     from backend.sports import sport_registry
31     from backend.providers import provider_registry
32     from backend.pipeline.detectors.base import DetectorRunner
33
34     logger = logging.getLogger(__name__)
35
36
37     def _fmt_ts(seconds: float) -> str:
38         seconds = max(0, int(seconds))
39         h, rem = divmod(seconds, 3600)
40         m, s = divmod(rem, 60)
41         return f"{h:02d}:{m:02d}:{s:02d}"
42
43
44     class TrajectoryHybridSegmenter(BasePlaySegmenter):
45         """Trajectory-detector hybrid: precise candidate rallies + AI semantic
46 boundaries."""
47
48         #: config section under `indexing` (velocity_threshold lives there), the
49         #: output subdir for trajectories, and the metadata detector-tag prefix.
50         family: str = ""
51         #: human-readable label used in log lines.
52         display_label: str = ""
53
54         def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
55 Any]]:
56             """Return (runner, detector-specific detection_params extras) for the default
57 (WSL) path."""
58             raise NotImplementedError
59
60         def _build_native_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner,
61 Dict[str, Any]]:
62             """Return (runner, extras) for the Linux-native (no-WSL) path. Only families
63 with a
64 native runner override this; the base refuses with a clear message (M3.5: WASB
65 only)."""
66             raise NotImplementedError(
67                 f"detector_impl='native' is not supported for the "
68                 f"{self.display_label or self.family or 'trajectory'!r} family ? only WASB
69 has a "
70                 f"Linux-native runner. Use detector_impl='auto' (WSL) or 'stub'.")
71
72         def _resolve_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner,
73 Dict[str, Any]]:
74             """Resolve the detector runner, honouring the CPU-stub and Linux-native
75 overrides.
76
77             ``RALLY_DETECTOR_IMPL`` env var (takes precedence) or
78             ``indexing.detector_impl``:
79             ``"stub"`` swaps in a deterministic CPU runner (no GPU/WSL) so the whole
80 pipeline
81 is regression-testable without a GPU ? "mock a GPU with a CPU"; ``"native"``
82 swaps
83 in the Linux-native WASB runner (no WSL/conda-activate; GPU box) via
84 ``_build_native_runner``. Anything else (``"auto"``) delegates to
85 ``_build_runner``

```

```

73         (the WSL runner) so the default production path is unchanged
74         (`docs/DECOUPLED_COMPUTE/`).
75         """
76         impl = (os.environ.get("RALLY_DETECTOR_IMPL") or idx_cfg.get("detector_impl") or
"auto").lower()
77         if impl == "stub":
78             from backend.pipeline.detectors.stub_runner import StubDetectorRunner,
StubConfig
79             logger.info("%s: detector_impl=stub ? CPU mock runner (no GPU/WSL).",
80                         self.display_label or "trajectory")
81             return StubDetectorRunner(StubConfig.from_indexing_cfg(idx_cfg)),
{"detector_impl": "stub"}
82         if impl in ("native", "native_wasb", "wasb_native"):
83             logger.info("%s: detector_impl=native ? Linux-native runner (no WSL).",
84                         self.display_label or "trajectory")
85             return self._build_native_runner(idx_cfg)
86         return self._build_runner(idx_cfg)
87
88     @staticmethod
89     def _probe_fps_width(video_path: str) -> tuple:
90         """Return (fps, frame_width) for a video, with sensible fallbacks."""
91         cap = cv2.VideoCapture(video_path)
92         fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
93         width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
94         cap.release()
95         return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
96
97     @staticmethod
98     def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
99         """Provider-reported exchange count, else a duration-based estimate.
100
101         One canonical implementation ? the twins had drifted (wasb relied on
102         ``int(None)`` raising into the fallback; same result, by accident).
103         """
104         shot_count = segment.get("shuttle_exchange_count")
105         if shot_count is None:
106             return max(3, int(duration / 0.8))
107         try:
108             return int(shot_count)
109         except (ValueError, TypeError):
110             return max(3, int(duration / 0.8))
111
112     # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)
113     -----
114     def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
115                                frame_width: float, video_path: str,
116                                idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
117         """Stats dict persisted into ``candidate_segments.stats`` for one window. The
BASE
118         returns exactly the 4 motion keys, so the trajectory twins persist byte-
identical
119         rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
120         features). ``points`` are the whole-video trajectory points
(TrajectoryPoint)."""
121         return {
122             "peak_velocity": cw["peak_velocity"],
123             "mean_velocity": cw["mean_velocity"],
124             "active_frame_count": cw["active_frame_count"],
125             "track_id_count": cw["track_id_count"],
126         }
127
128     def _select_for_handoff(self, windows: List[Dict[str, Any]],
129                            window_stats: List[Dict[str, Any]],
130                            idx_cfg: Dict[str, Any]) -> List[int]:
131         """Indices of the candidate windows that proceed to the AI handoff (and thus may

```

```

131         become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
132         ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
133         Persisted candidates are NOT affected ? they remain the full superset for
offline
134         re-scoring."""
135         return list(range(len(windows)))
136
137     def process_video(self, video_id: str, video_path: str, # type: ignore[override]
138                       player_pool: Optional[List[str]] = None,
139                       max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
140         # override-ignore: the ABC declares the Result contract (Success|Failure)
141         # but every live segmenter still returns a bare list ? the documented
142         # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
143         label = self.display_label
144         # --- Sport profile + AI provider wiring (identical across segmenters) ---
145         sport_name = self.config.get("sport", "badminton")
146         try:
147             sport_profile = sport_registry.get(sport_name)
148         except KeyError as e:
149             logger.error(str(e))
150             return []
151
152         idx_cfg = self.config.get("indexing", {})
153         # Fully-local, NO-AI mode (default OFF): the trajectory candidate windows become
the
154         # rallies directly ? Phase 3's AI handoff is skipped, so no provider / API key
is
155         # needed and nothing is uploaded. Used to run the local detector standalone
(e.g. to
156         # measure it against the Gemini path, or to avoid the cloud entirely). With
157         # FusionSegmenter the windows are still Gate-A/B-filtered via
`_select_for_handoff`.
158         skip_ai = bool(idx_cfg.get("skip_ai_handoff", False))
159         provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
160         if skip_ai:
161             model_name = "local-noai"
162             logger.info("%s in FULLY-LOCAL mode (skip_ai_handoff=True): candidate
windows will "
163                         "be emitted as rallies; no %s handoff, no API key needed.",
164                         label, provider_name)
165         else:
166             model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
167             api_key_env_var = f"{provider_name.upper()}_API_KEY"
168             api_key = os.environ.get(api_key_env_var)
169             if not api_key:
170                 from backend.pipeline.segmenters._keyhint import api_key_hint
171                 logger.error(
172                     f"{api_key_env_var} environment variable is not set! "
173                     f"To run the {provider_name} handoff, set your API key. "
174                     + api_key_hint(api_key_env_var)
175                 )
176             return []
177         try:
178             provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
179         except KeyError as e:
180             logger.error(str(e))
181             return []
182
183         video_duration = get_video_duration(video_path)
184         if video_duration <= 0:
185             logger.error("Invalid video duration.")
186             return []

```

```

187         fps, frame_width = self._probe_fps_width(video_path)
188
189         # --- Runner config + windowing thresholds ---
190         # _resolve_runner honours the CPU-stub override (RALLY_DETECTOR_IMPL /
191         # indexing.detector_impl="stub") so the pipeline runs GPU/WSL-free; otherwise
192         # it delegates to the subclass _build_runner (the real WSL/native runner).
193         runner, runner_params = self._resolve_runner(idx_cfg)
194
195         velocity_thresh = idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
196         merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
197         min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
198         # Bounded-windowing over-merge fix (W1): 0 = OFF (unchanged). See
IndexingConfig.
199         max_window_duration = idx_cfg.get("max_window_duration", 0.0)
200         window_min_split_gap = idx_cfg.get("window_min_split_gap", 0.5)
201         window_hard_cap = idx_cfg.get("window_hard_cap", False)
202         window_inactivity_end = idx_cfg.get("window_inactivity_end", 0.0)
203         inactivity_rally_thresh = idx_cfg.get("inactivity_rally_thresh", 0.08)
204         inactivity_min_density = idx_cfg.get("inactivity_min_density", 0.34)
205         window_blob_fallback = idx_cfg.get("window_blob_fallback", False)
206         window_blob_factor = idx_cfg.get("window_blob_factor", 4.0)
207         handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
208         ai_max_workers = idx_cfg.get("ai_max_workers", 5)
209         log_candidates = idx_cfg.get("log_yolo_candidates", True)
210         store_candidates = idx_cfg.get("store_yolo_candidates", True)
211
212         detection_params = {
213             "detector": self.name,
214             "velocity_thresh": velocity_thresh,
215             "merge_gap": merge_gap,
216             "min_action_window_duration": min_window_duration,
217             **runner_params,
218         }
219
220         # --- Phase 1: env healthcheck (fail fast with a clear message) ---
221         ok, msg = runner.healthcheck()
222         if not ok:
223             logger.error("%s detector environment not ready: %s", label, msg)
224             return []
225         logger.info("%s detector env OK: %s", label, msg)
226
227         # --- Phase 2: trajectory -> candidate rally windows ---
228         # Trajectory detectors process the whole video in one pass (they carry
229         # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
230         t_start = time.time()
231         out_dir = os.path.join("output", self.family)
232         csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
233         if not csv_path:
234             logger.error("%s produced no trajectory; aborting.", label)
235             return []
236
237         points = runner.parse_trajectory_csv(csv_path)
238         logger.info("Parsed %d trajectory points (%d visible).",
239                     len(points), sum(1 for p in points if p.visible))
240
241         all_candidate_windows = runner.trajectory_to_action_windows(
242             points, fps=fps, frame_width=frame_width, chunk_start=0.0,
243             velocity_thresh=velocity_thresh, merge_gap=merge_gap,
244             min_window_duration=min_window_duration, chunk_index=0,
245             max_window_duration=max_window_duration, min_split_gap=window_min_split_gap,
246             window_hard_cap=window_hard_cap,
247             window_inactivity_end=window_inactivity_end,
248             inactivity_rally_thresh=inactivity_rally_thresh,
249             inactivity_min_density=inactivity_min_density,
250             window_blob_fallback=window_blob_fallback,

```

```

251         window_blob_factor=window_blob_factor,
252     )
253     logger.info("Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
254                 label, time.time() - t_start, len(all_candidate_windows))
255
256     if log_candidates:
257         for cw in all_candidate_windows:
258             logger.info(f"[{label} rally]
259             {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
260             f"({cw['end'] - cw['start']:.1f}s) |
261             frames={cw['active_frame_count']} "
262             f"peak_v={cw['peak_velocity']:.3f}
263             mean_v={cw['mean_velocity']:.3f}")
264
265     # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
266     fusion
267     # segmenter adds net_crossings + person-cue features. Computed once, reused for
268     both
269     # persistence and the handoff gate below.
270     window_stats = [
271         self._enrich_candidate_stats(cw, points, fps, frame_width, video_path,
272         idx_cfg)
273         for cw in all_candidate_windows
274     ]
275
276     # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
277     candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
278     if store_candidates:
279         for i, cw in enumerate(all_candidate_windows):
280             candidate_ids[i] = self.db.add_candidate_segment(
281                 video_id, detector=self.name,
282                 start_time=cw["start"], end_time=cw["end"],
283                 chunk_index=cw["chunk_index"], confidence=min(1.0,
284                 cw["peak_velocity"]),
285                 stats=window_stats[i], detection_params=detection_params,
286             )
287
288     if not all_candidate_windows:
289         return []
290
291     # --- Phase 3: AI provider handoff over padded candidate clips ---
292     # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
293     Persisted
294     # candidates above stay the full superset ? only the served play_segments are
295     gated.
296     handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
297     idx_cfg)
298
299     ai_segments: List[dict] = []
300     if skip_ai:
301         # Fully-local: the selected candidate windows ARE the rallies ? emit them
302         verbatim
303         # (no clip extraction, no upload). Boundaries are the local detector's; shot
304         count
305         # falls back to the duration-based estimate (no AI exchange count).
306         ai_segments = [
307             {
308                 "start_time": all_candidate_windows[wi]["start"],
309                 "end_time": all_candidate_windows[wi]["end"],
310                 "shuttle_exchange_count": None,
311                 "shot_count_confidence": "LocalNoAI",
312                 "_candidate_id": candidate_ids[wi],
313             }
314             for wi in handoff_indices
315         ]

```

```

304         logger.info("Phase 3 SKIPPED (fully-local): emitted %d candidate windows as
rallies "
305                     "(no AI handoff).", len(ai_segments))
306     else:
307         handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
308         os.makedirs(handoff_dir, exist_ok=True)
309         logger.info("Phase 3: %s validation on %d candidate windows...",
310                     provider_name, len(handoff_indices))
311
312     def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
313         w_start = max(0, window["start"] - handoff_padding)
314         w_end = min(video_duration, window["end"] + handoff_padding)
315         w_dur = w_end - w_start
316         w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
317         if not extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True):
318             return []
319         prompt = sport_profile.get_prompt(chunk_duration=w_dur)
320         segments, _ = provider.analyze_video(w_path, prompt,
chunk_duration=w_dur,
321                                             label=f"Handoff {wi+1}")
322         valid = []
323         for seg in segments:
324             try:
325                 seg["start_time"] = float(seg["start_time"]) + w_start
326                 seg["end_time"] = float(seg["end_time"]) + w_start
327                 seg["_candidate_id"] = candidate_ids[wi]
328                 valid.append(seg)
329             except (ValueError, TypeError, KeyError) as e:
330                 logger.warning("Dropping segment with unparseable timestamps: %s
? %s", seg, e)
331         try:
332             os.remove(w_path)
333         except OSError:
334             pass
335         return valid
336
337     with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
338         futures = [executor.submit(process_candidate_window, wi,
all_candidate_windows[wi])
339                     for wi in handoff_indices]
340         for future in futures:
341             ai_segments.extend(future.result())
342         try:
343             os.rmdir(handoff_dir)
344         except OSError:
345             pass
346         logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
len(ai_segments))
347
348     # --- Phase 4: DB population + candidate linking ---
349     segments_data = []
350     for segment in ai_segments:
351         try:
352             start = float(segment["start_time"])
353             end = float(segment["end_time"])
354             dur = end - start
355             metadata = {
356                 "shot_count": self._shot_count_from(segment, dur),
357                 "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
358                 "detector": f"{self.family}-hybrid-{model_name}",
359             }
360             segment_id = self.db.add_play_segment(video_id, start, end, None, None,

```

```

metadata)
361         if segment_id:
362             candidate_id = segment.get("_candidate_id")
363             if candidate_id is not None:
364                 self.db.link_candidate_to_segment(candidate_id, segment_id)
365             segments_data.append({
366                 "id": segment_id, "start_time": start, "end_time": end,
367                 "duration": dur, "metadata": metadata,
368             })
369         except Exception as e:
370             logger.warning("Skipping invalid segment struct: %s", e)
371
372     return segments_data
373

```

6. assistant (2026-06-20T09:03:10.760Z)

Now I've confirmed `\_resolve\_runner` calls `\_build\_native\_runner` which raises `NotImplementedError` for non-WASB families. Let me check the worker tests and the existing pattern to confirm whether the finding is accurate, and check what the test file expects.

7. user (2026-06-20T09:03:11.467Z)

```

1     """Worker dispatcher: pull (storage) ? run pipeline ? push (storage).
2
3     `cmd_infer` is exercised end-to-end over the LOCAL storage backend with the validator +
4     segmenter mocked, so it needs no real video/GPU/network. The real S3 round-trip is
validated
5     on the droplet against MinIO.
6     """
7     import json
8
9     import pytest
10
11     from backend.config import load_config
12     from backend.worker import __main__ as wmain
13
14
15     # ----- pure helpers
16
17     @pytest.mark.parametrize("raw,expected", [
18         ("true", True), ("False", False), ("3", 3), ("2.5", 2.5), ("wasb_hybrid",
"wasb_hybrid"),
19     ])
20     def test_coerce(raw, expected):
21         assert wmain._coerce(raw) == expected and type(wmain._coerce(raw)) is type(expected)
22
23
24     def test_apply_overrides_on_typed_config():
25         cfg = _fresh_config()
26         wmain._apply_overrides(cfg, ["indexing.skip_ai_handoff=true",
"indexing.min_segment_duration=1.5",
27                                     "sport=tennis"])
28         assert cfg.indexing.skip_ai_handoff is True
29         assert cfg.indexing.min_segment_duration == 1.5
30         assert cfg.sport == "tennis"
31
32
33     def test_apply_overrides_rejects_bad_format():
34         cfg = _fresh_config()
35         with pytest.raises(ValueError):
36             wmain._apply_overrides(cfg, ["no_equals_sign"])
37
38
39     def test_write_intervals_csv(tmp_path):

```

```

40         segs = [{"start_time": 2.0, "end_time": 5.0, "shot_count": 4,
"shot_count_confidence": "LocalNoAI"},
41                 {"start_time": 9.0, "end_time": 12.5}]
42         out = tmp_path / "intervals.csv"
43         wmain._write_intervals_csv(segs, str(out))
44         lines = out.read_text().splitlines()
45         assert lines[0] == "start,end,shot_count,ending_reason"
46         assert lines[1].startswith("2.000,5.000,4,")
47         assert lines[2].startswith("9.000,12.500,,")
48
49
50     def test_write_report_json(tmp_path):
51         out = tmp_path / "report.json"
52         wmain._write_report_json("vid1", [{"start_time": 1.0, "end_time": 2.0}], "success",
str(out))
53         data = json.loads(out.read_text())
54         assert data["video_id"] == "vid1" and data["segment_count"] == 1
55         assert data["segments"][0] == {"start": 1.0, "end": 2.0}
56
57
58     def test_parser_infer_accepts_set_and_uris():
59         args = wmain.build_parser().parse_args([
60             "infer", "--video-uri", "s3://b/v.mp4", "--out-uri", "s3://b",
61             "--set", "indexing.skip_ai_handoff=true", "--set", "sport=tennis"])
62         assert args.cmd == "infer" and args.video_uri == "s3://b/v.mp4"
63         assert args.set == ["indexing.skip_ai_handoff=true", "sport=tennis"]
64
65
66     # ----- cmd_infer (local,
mocked)
67
68     class _OKResult:
69         passed = True
70         validator_name = "fake"
71         message = "ok"
72
73         def model_dump(self):
74             return {"validator_name": "fake", "passed": True, "message": "ok"}
75
76
77     class _FailResult(_OKResult):
78         passed = False
79         message = "too blurry"
80
81
82     class _FakeSeg:
83         def __init__(self, db, config):
84             pass
85
86         def process_video(self, video_id, path, max_frames=None):
87             return [
88                 {"start_time": 2.0, "end_time": 5.0, "shot_count": 4,
"shot_count_confidence": "LocalNoAI"},
89                 {"start_time": 9.0, "end_time": 12.0, "shot_count": 6,
"shot_count_confidence": "LocalNoAI"},
90             ]
91
92
93     def _fresh_config():
94         # Load defaults without touching any cwd config.json.
95         import tempfile
96         p = tempfile.NamedTemporaryFile("w", suffix=".json", delete=False)
97         p.write("{}")
98         p.close()
99         return load_config(p.name)

```



```

100
101
102 @pytest.fixture
103 def mocked_pipeline(monkeypatch):
104     monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vid123")
105     monkeypatch.setattr(wmain.validator_registry, "run_all_with_context",
106                         lambda path, cfg: ([_OKResult()], {}))
107     monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _FakeSeg)
108     monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
109
110
111 def test_cmd_infer_local_roundtrip(tmp_path, mocked_pipeline):
112     video = tmp_path / "in.mp4"
113     video.write_bytes(b"FAKEVIDEO")
114     cfg = tmp_path / "config.json"
115     cfg.write_text("{}")
116     out = tmp_path / "bucket"
117     scratch = tmp_path / "scratch"
118
119     rc = wmain.main([
120         "infer", "--video-uri", str(video), "--out-uri", str(out),
121         "--config", str(cfg), "--scratch", str(scratch),
122         "--segmenter", "wasb_hybrid", "--set", "indexing.skip_ai_handoff=true",
123     ])
124     assert rc == 0
125     intervals = out / "outputs" / "vid123" / "intervals.csv"
126     report = out / "outputs" / "vid123" / "report.json"
127     assert intervals.exists() and report.exists()
128     rows = intervals.read_text().splitlines()
129     assert len(rows) == 3 # header + 2 rallies
130     assert json.loads(report.read_text())["segment_count"] == 2
131
132
133 def test_cmd_infer_validation_failure_returns_2(tmp_path, monkeypatch):
134     monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidF")
135     monkeypatch.setattr(wmain.validator_registry, "run_all_with_context",
136                         lambda path, cfg: ([_FailResult()], {}))
137     monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
138     video = tmp_path / "in.mp4"
139     video.write_bytes(b"X")
140     cfg = tmp_path / "config.json"
141     cfg.write_text("{}")
142     rc = wmain.main(["infer", "--video-uri", str(video), "--out-uri", str(tmp_path /
143 "o"),
144                     "--config", str(cfg), "--scratch", str(tmp_path / "s"))]
145     assert rc == 2
146
147 # --- cmd_train: pull labels -> synth trajectories -> manifest -> harness -> push bundle
148 ---
149 def test_cmd_train_orchestration(tmp_path, monkeypatch):
150     import os
151     import subprocess
152
153     labels_dir = tmp_path / "corpus" / "labels"
154     labels_dir.mkdir(parents=True)
155     header = "rally_number,start_time,end_time,ending_reason,sport,shots_count\n"
156     (labels_dir / "v1.rallies.csv").write_text(header +
157 "1,2.0,5.0,winner,badminton,\n2,9.0,12.0,let,badminton,\n")
158     (labels_dir / "v2.rallies.csv").write_text(header + "1,3.0,6.0,winner,badminton,\n")
159     out_dir = tmp_path / "out"
160     scratch = tmp_path / "scratch"
161     cfg = tmp_path / "config.json"
162     cfg.write_text("{}")

```

```

162
163     # Mock the harness subprocess: write a fake versioned bundle at --out/--version.
164     def fake_run(cmd, *a, **k):
165         out = cmd[cmd.index("--out") + 1]
166         ver = cmd[cmd.index("--version") + 1]
167         bdir = os.path.join(out, ver)
168         os.makedirs(bdir, exist_ok=True)
169         for fn in ("weights.json", "manifest.json"):
170             with open(os.path.join(bdir, fn), "w") as f:
171                 f.write("{}")
172         return subprocess.CompletedProcess(cmd, 0)
173     monkeypatch.setattr(subprocess, "run", fake_run)
174
175     rc = wmain.main(["train", "--corpus-uri", str(tmp_path / "corpus"), "--out-uri",
176 str(out_dir),
177                     "--version", "0.0.1", "--config", str(cfg), "--scratch",
178 str(scratch)])
179     assert rc == 0
180     # bundle pushed to <out-uri>/weights/<version>/
181     assert (out_dir / "weights" / "0.0.1" / "weights.json").exists()
182     assert (out_dir / "weights" / "0.0.1" / "manifest.json").exists()
183     # the manifest fed to the harness enumerated both labeled videos with a trajectory
184     each
185         specs = json.loads((scratch / "manifest.json").read_text())
186         assert len(specs) == 2
187         assert all("trajectory" in s and "labels" in s and "fps" in s for s in specs)
188
189     def test_parser_train_detector_args():
190         args = wmain.build_parser().parse_args([
191             "train", "--corpus-uri", "gcs://b", "--out-uri", "gcs://b", "--version",
192 "1.0.0",
193             "--trajectory-source", "detector", "--segmenter", "fusion_hybrid"])
194         assert args.trajectory_source == "detector" and args.segmenter == "fusion_hybrid"
195
196     def test_cmd_train_detector_source_runs_real_extract_via_stub(tmp_path, monkeypatch):
197         """--trajectory-source detector pulls videos + runs the resolved DetectorRunner.
198 With
199         RALLY_DETECTOR_IMPL=stub the *wiring* is exercised GPU-free (the stub IS the
200 runner)."""
201         import os
202         import subprocess
203
204         monkeypatch.setenv("RALLY_DETECTOR_IMPL", "stub") # resolve a CPU stub runner (no
GPU/WSL)
205         corpus = tmp_path / "corpus"
206         labels_dir = corpus / "labels"
207         videos_dir = corpus / "videos"
208         labels_dir.mkdir(parents=True)
209         videos_dir.mkdir(parents=True)
210         header = "rally_number,start_time,end_time,ending_reason,sport,shots_count\n"
211         (labels_dir / "v1.rallies.csv").write_text(header + "1,2.0,5.0,winner,badminton,\n")
212         (labels_dir / "v2.rallies.csv").write_text(header + "1,3.0,6.0,winner,badminton,\n")
213         (videos_dir / "v1.mp4").write_bytes(b"FAKEVIDE01")
214         (videos_dir / "v2.mp4").write_bytes(b"FAKEVIDE02")
215         out_dir = tmp_path / "out"
216         scratch = tmp_path / "scratch"
217         cfg = tmp_path / "config.json"
218         cfg.write_text("{}")
219
220     def fake_run(cmd, *a, **k):
221         out = cmd[cmd.index("--out") + 1]
222         ver = cmd[cmd.index("--version") + 1]
223         bdir = os.path.join(out, ver)

```

```

220         os.makedirs(bdir, exist_ok=True)
221         for fn in ("weights.json", "manifest.json"):
222             with open(os.path.join(bdir, fn), "w") as f:
223                 f.write("{}")
224         return subprocess.CompletedProcess(cmd, 0)
225 monkeypatch.setattr(subprocess, "run", fake_run)
226
227     rc = wmain.main(["train", "--corpus-uri", str(corpus), "--out-uri", str(out_dir),
228                     "--version", "0.0.2", "--config", str(cfg), "--scratch",
str(scratch),
229                     "--trajectory-source", "detector", "--segmenter", "wasb_hybrid"])
230     assert rc == 0
231     specs = json.loads((scratch / "manifest.json").read_text())
232     assert len(specs) == 2
233     # Real-extract path: each spec points at an on-disk trajectory the runner actually
wrote.
234     assert all(os.path.exists(s["trajectory"]) for s in specs)
235     assert all(s["trajectory"].endswith("_stub.csv") for s in specs) # the stub
runner's output
236

```

8. assistant (2026-06-20T09:03:16.073Z)

Now let me verify the key claim. The finding says ``config.get("indexing", {})`` is called on line 183. But ``config`` here is a typed Pydantic config object (from ``load_config``), not a dict. Let me check whether ``config.get`` even works. Let me look at how ``_resolve_train_detector`` is called - line 229 passes ``config`` (the typed config), and inside ``_resolve_train_detector``, line 178 does ``seg_cls(None, config)`` and line 183 does ``config.get("indexing", {})``.

Let me check the config object and whether ``get`` works on it.

9. user (2026-06-20T09:03:16.925Z)

```

1     """Pydantic models for the application configuration.
2
3     This module defines the canonical, validated, typed configuration for the
4     Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6     The models support graceful migration from old config.json files via extra="allow".
7     """
8
9     from __future__ import annotations
10
11     from typing import Any, List, Literal, Optional
12
13     from pydantic import BaseModel, Field, field_validator, model_validator
14
15     Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
18     class ValidationRelevanceConfig(BaseModel):
19         court_green_lower: list[int] = Field(default=[35, 40, 40])
20         court_green_upper: list[int] = Field(default=[85, 255, 255])
21         court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23
24     class ValidationConfig(BaseModel):
25         min_resolution_width: int = 1280
26         min_resolution_height: int = 720
27         min_fps: float = 29.9
28         min_blur_threshold: float = 20.0
29         max_scene_cuts_per_minute: float = 0.5
30         relevance: ValidationRelevanceConfig =
Field(default_factory=ValidationRelevanceConfig)
31

```

```

32
33 class TrackNetConfig(BaseModel):
34     wsl_distro: str = "Ubuntu"
35     repo_dir: str = "~/models/TrackNetV4"
36     conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
37     conda_env: str = "TrackNetV4"
38     weights_path: str = ""
39     queue_length: int = 5
40     stage_in_wsl: bool = True
41     wsl_stage_dir: str = "~/clips"
42     timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
43     velocity_threshold: float = 0.02
44     # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
45     # decoded frame cache) are deleted automatically ? they are pure intermediates,
46     # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
47     # true only for debugging. On FAILURE they are always kept so a re-run can resume.
48     keep_frames: bool = False
49     # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB.
50     # 0 = disabled.
51     min_free_gb: float = 0.0
52
53
54 class WasbIndexConfig(BaseModel):
55     """Typed config for the live WASB shuttle substrate (indexing.wasb).
56
57     Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
58     these knobs actually take effect ? previously this whole block was silently dropped
59     because nested config sections defaulted to extra='ignore'.
60     """
61     wsl_distro: str = "Ubuntu"
62     repo_dir: str = "~/models/WASB-SBDT"
63     conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
64     conda_env: str = "wasb"
65     weights_path: str = "~/models/WASB-
66 SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
67     sport: str = "badminton"
68     wsl_stage_dir: str = "~/clips"
69     timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
70     velocity_threshold: float = 0.02
71     # --- Linux-native runner only (detector_impl='native'); ignored by the WSL runner.
72     ---
73     # Env vars override these (WASB_DEVICE / WASB_PYTHON) for a config-free GPU box.
74     device: str = "cuda" # torch device for the native runner: cuda | mps | cpu
75     python_bin: str = "python" # the `wasb` conda env python (invoked by path, no
76 `conda activate`)
77     # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
78     # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
79     regenerable
80     # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
81     resumes.
82     keep_frames: bool = False
83     # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB. 0 =
84     disabled.
85     min_free_gb: float = 0.0
86     # FAST PATH (default OFF until owner-verified side-by-side): decode the staged video
87     # on the fly and feed frames straight to the detector, skipping the slow per-frame
88     PNG
89     # extraction that dominates a long clip (~46k PNGs for a 12-min 1080p60 video, which
90     # overran the 30-min timeout). Output is bit-identical to the PNG path (PNG is
91     lossless),
92     # so this is purely a speed/disk win; kept opt-in until the side-by-side confirms
93     it.
94     stream_video: bool = False

```

```

88     class FusionConfig(BaseModel):
89         """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
90 P2).
91
92         Every default reproduces control behaviour: the fusion segmenter persists the
93         shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']``
94         is
95         true ? the person-cue features, but it does NOT gate the served handoff unless a
96         threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court
97         mask
98         is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-
99         only);
100         supplied ? off-court persons (spectators / adjacent courts) are excluded.
101         """
102         # Which trajectory substrate seeds the windows (shuttle stays primary).
103         substrate: Literal["wasb", "tracknet"] = "wasb"
104         # Windowing velocity threshold for the fusion family (the engine reads
105         # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
106         # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
107         velocity_threshold: float = 0.02
108         # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
109         # enabled ? so the default path never imports torch / touches the GPU.
110         cue_flags: dict[str, bool] = Field(default_factory=dict)
111         # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
112         handoff.
113         # 0 = OFF (no gating; persisted candidates are always the full superset for offline
114         re-scoring).
115         min_crossings: int = Field(default=0, ge=0)
116         # Served Gate B: when the person cue is on, drop windows with median in-court
117         players
118         # < this. 0 = OFF.
119         min_players: int = Field(default=0, ge=0)
120         # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
121         court_polygon: Optional[list[list[float]]] = None
122         # Net line for the person two-sided-motion split (person features only ? the shuttle
123         # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
124         net_axis: Literal["x", "y"] = "y"
125         net_position: float = Field(default=0.5, ge=0.0, le=1.0)
126
127     class IndexingConfig(BaseModel):
128         default_segmenter: str = "yolo_hybrid"
129         use_gpu: bool = True
130         # Detector backend for the trajectory hybrids (wasb/tracknet). "auto" = the real
131         # WSL runner (default, production); "stub" = a deterministic CPU mock (no GPU/WSL)
132         # so the whole pipeline is regression-testable on a CPU box / CI; "native" = the
133         # Linux-native WASB runner (no WSL/conda-activate; a GPU box ? M3.5). The
134         # RALLY_DETECTOR_IMPL env var overrides this. See
135         pipeline/detectors/{stub,native_wasb}_runner.py.
136         detector_impl: str = "auto"
137         motion_threshold: float = 0.08
138         min_segment_duration: float = 3.0
139         dead_time_merge_gap: float = 2.0
140         chunk_duration: float = 90.0
141         chunk_overlap: float = 10.0
142         detector_model: str = "fasterrcnn_resnet50_fpn"
143         detector_score_threshold: float = 0.5
144         yolo_velocity_threshold: float = 0.15
145         yolo_frame_skip: int = 3
146         yolo_tracker: str = "bytetrack.yaml"
147         yolo_prefetch_workers: int = 2
148         min_action_window_duration: float = 2.0
149         # BOUNDED WINDOWING (over-merge fix W1, RALLY_QUALITY_RESEARCH.md §6). 0 = OFF; > 0
150         = a window
151         # longer than this many seconds is recursively split at its largest internal

```

```

inactivity gap (?
144      # ``window_min_split_gap`` s ? the most likely rally boundary), so one window can't
swallow
145      # several rallies + the dead time between them. Never merges, only cuts (recall
can't drop).
146      # *** R1 MILESTONE DEFAULT-ON (cap=6): the smallest safe local-windowing flip.
Measured n=13
147      # golden, R2 tolF1: baseline?milestone (cap=6 + R4 below) ?+0.222, sign 12/0,
p=0.0005; cap=6
148      # beats cap=12 ?+0.110, sign 12/0; net over-seg guard PASS (mean merge_rate
0.651?0.161). ***
149      max_window_duration: float = 6.0
150      window_min_split_gap: float = 0.5 # min internal gap (s) that qualifies as a split
point
151      # HARD-CAP fallback for W1: when True, an over-long window with NO internal
inactivity gap
152      # (a continuously-active trajectory ? e.g. the real-footage mahadevpura-1 174s blob)
is
153      # chopped at its midpoint until ? max_window_duration, making the cap a true upper
bound.
154      # Only cuts (recall can't drop). OFF by default until measured/promoted.
155      window_hard_cap: bool = False
156      # R4 rally-END inactivity trim (s, 0 = OFF). After a rally the shuttle keeps moving
slowly
157      # (picked up / walked) above velocity_thresh, so the window over-extends its END
(the measured
158      # [?end] gap vs golden). Trim the post-rally tail back to the last frame whose
trailing
159      # window stays dense with FAST motion (velocity > inactivity_rally_thresh). Only
cuts.
160      # *** R1 MILESTONE DEFAULT-ON (1.5/0.20/0.30): on top of cap=6, R4 adds a small but
significant
161      # end-precision gain ? ?tolF1 +0.040, sign 9/1/3, p=0.022, [?end] 4.96?3.31s (n=13).
The W1 cap
162      # is the dominant lever; R4 is the marginal increment. See QUALITY_ITERATIONS.md
"R1". ***
163      window_inactivity_end: float = 1.5
164      inactivity_rally_thresh: float = 0.20
165      inactivity_min_density: float = 0.30
166      # R5 blob-fallback (default OFF). LAST-RESORT splitter for the continuously-active
blob: after
167      # the W1 gap-split AND the R4 inactivity trim have each had their chance, a group
STILL longer
168      # than window_blob_factor × max_window_duration (no internal gap, no rally-end
signal ? e.g.
169      # mahadevpura-1's ~65s residual ? 11× a 6s cap) is midpoint-chopped to ? the cap as
a FORCED cut
170      # (logged + flagged on the window) so the degenerate single-window outcome is
avoided and the
171      # clip is surfaced as needing rally-STATE cues, not a bigger cap. Differs from
window_hard_cap
172      # (which chops BEFORE R4 and over-segments normal clips): blob-fallback runs AFTER
R4 and only
173      # force-cuts the genuine blob (the factor gate). Only cuts (recall can't drop).
174      window_blob_fallback: bool = False
175      # Magnitude gate that keeps R5 surgical: only a group longer than this ×
max_window_duration is a
176      # blob (a normal long rally ~1-2× the cap is left alone; the mahadevpura-1 residual
is ~11×).
177      # Default 4.0 is do-no-harm on the golden corpus (rescues only the continuously-
active
178      # clips, every other clip a no-op within noise) ? see docs/QUALITY_ITERATIONS.md
"R5".
179      window_blob_factor: float = 4.0
180      log_yolo_candidates: bool = True

```

```

181     store_yolo_candidates: bool = True
182     ai_handoff_padding: float = 2.0
183     ai_max_workers: int = 5
184     # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
185     # chunks of a high-bitrate (4K) source blow past the provider upload cap
186     # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
187     # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
188     # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
189     # Matches gemini_refine's --clip-scale-width default.
190     ai_chunk_scale_width: int = 1280
191     # FULLY-LOCAL mode (default OFF): when true, the trajectory-hybrid segmenters
192     # (wasb_hybrid / tracknet_hybrid / fusion_hybrid) skip the Phase-3 AI handoff
entirely and
193     # emit their local candidate windows AS the rallies ? no provider, no API key,
nothing
194     # uploaded. Lets the local detector run standalone (e.g. to benchmark it against the
Gemini
195     # path, or to operate with zero cloud dependency). The pure `gemini` segmenter
ignores this.
196     skip_ai_handoff: bool = False
197     # Optional debug knob: trim every video to the first N seconds before processing.
198     debug_duration: Optional[float] = None
199
200     tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
201     wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
202     fusion: FusionConfig = Field(default_factory=FusionConfig)
203
204     @field_validator("default_segmenter")
205     @classmethod
206     def segmenter_must_be_registered(cls, v: str) -> str:
207         # Registry check happens at runtime in main/segmenter selection.
208         # Here we just allow any string for forward compatibility.
209         return v
210
211     @model_validator(mode="after")
212     def _chunk_step_must_be_positive(self) -> "IndexingConfig":
213         # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
positive
214         # step makes `while t < duration: t += step` loop forever, growing memory before
any
215         # GPU work. Reject the bad config at load time instead.
216         if self.chunk_overlap >= self.chunk_duration:
217             raise ValueError(
218                 f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
"
219                 f"({self.chunk_duration}); otherwise chunking never advances."
220             )
221         return self
222
223
224     class OutputConfig(BaseModel):
225         default_highlights_name: str = "highlights.mp4"
226         db_name: str = "sports_indexer.db"
227         # Directory where the DB, highlight reels, and processed clips are written.
228         # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
229         output_dir: str = "output"
230
231
232     class WatermarkConfig(BaseModel):
233         """Branding overlay burned into each highlight clip during the per-clip re-encode
234         (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
235         (under `stitching.watermark` in config.json) to turn it off. The text is fully
236         configurable. The concat stage stays stream-copy (-c copy)."""
237         enabled: bool = True
238         text: str = "Generated by Khelsutra.guru"

```

```

239     logo_path: str = ""          # optional transparent PNG overlay; "" = no logo
240     font_path: str = ""          # optional .ttf; "" = use the fontconfig `font` family
below
241     font: str = "Sans"           # fontconfig family used when font_path is empty
242     font_size_ratio: float = Field(default=0.035, gt=0.0, le=0.5) # text height as a
fraction of frame height
243     opacity: float = Field(default=0.85, ge=0.0, le=1.0)
244     box: bool = True             # faint backing box behind the text for legibility
245     margin: int = Field(default=24, ge=0) # px inset from the chosen corner
246     position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = "bottom-
right"
247
248
249     class StitchingConfig(BaseModel):
250         begin_padding: float = 0.5
251         end_padding: float = 0.5
252         use_cache: bool = False
253         min_shot_count: int = 1
254         # Long-rally reel filter (seconds; 0 = OFF). Drop detected rally pairs whose
duration
255         # (end ? start) is below this from the highlight reel, BEFORE compiling. A cleaner
256         # reel-selection knob than `min_shot_count`: shot_count is unlabeled / "Unknown" on
the
257         # fully-local no-AI path, whereas duration is derived from the rally's own
start/end. The
258         # local detector over-fires and its false positives are mostly SHORT (motion blips,
259         # background-court fragments), so this keeps the reel to the eye-catching long
rallies.
260         # OFF by default ? the detector output is unchanged, only which rallies reach the
reel.
261         min_rally_duration: float = Field(default=0.0, ge=0.0)
262         watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)
263
264
265     class LoggingConfig(BaseModel):
266         """Logging knobs for the run entrypoints (see backend/utils/logging_setup.py)."""
267         # Third-party HTTP/RPC client loggers (httpx, httpcore, urllib3, google-genai,
268         # google.auth, grpc) emit one INFO line per request ? dozens per Gemini run, which
269         # buries our own [rally]/[compile] lines in run.log during manual QA. Default raises
270         # them to WARNING; set true to restore full chatter for request-flow debugging.
271         verbose_http: bool = False
272
273
274     class SecurityConfig(BaseModel):
275         # When set, /api/process video_path must resolve under this directory (hosted/tenant
276         # mode). Default None preserves the single-user local 'any local file' workflow.
277         allowed_video_root: Optional[str] = None
278
279         # --- Chunked upload (B2, PR-2) ---
280         # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
281         # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
282         # contain upload_dir or /api/process will reject the uploaded paths.
283         upload_dir: str = "output/uploads"
284         # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
285         # free-tier edge proxies cap request bodies there (HOSTING_PLAN $2).
286         max_upload_mb: int = 4096
287         max_part_mb: int = 95
288         allowed_upload_extensions: List[str] = ["mp4", "mov"]
289
290
291     class StorageConfig(BaseModel):
292         """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
293         behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + DO
Spaces
294         + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = Google

```



```

Drive
295         (follow-up). Per-backend settings are loose dicts passed to the backend constructor
?
296         **secrets are never stored here**, only endpoints / regions / file paths / source-
selectors
297         (boto3/google auth chains supply credentials). See ``backend/storage/``.
298         backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
299         # Output root for produced artifacts. "" => fall back to output.output_dir (non-
breaking).
300         output_root: str = ""
301         local: dict[str, Any] = Field(default_factory=dict)
302         s3: dict[str, Any] = Field(default_factory=dict)
303         gcs: dict[str, Any] = Field(default_factory=dict)
304         gdrive: dict[str, Any] = Field(default_factory=dict)
305
306
307     class AppConfig(BaseModel):
308         """Root configuration object.
309
310         All runtime configuration should be accessed via an instance of this class
311         (or the loader) rather than raw dict lookups.
312         """
313
314         sport: Sport = "badminton"
315         ai_provider: str = "gemini"
316         # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
flagship;
317         # the alias never deprecates underneath us. Eval reports record the run date
alongside.
318         ai_model: str = "gemini-flash-latest"
319
320         validation: ValidationConfig = Field(default_factory=ValidationConfig)
321         indexing: IndexingConfig = Field(default_factory=IndexingConfig)
322         output: OutputConfig = Field(default_factory=OutputConfig)
323         stitching: StitchingConfig = Field(default_factory=StitchingConfig)
324         security: SecurityConfig = Field(default_factory=SecurityConfig)
325         logging: LoggingConfig = Field(default_factory=LoggingConfig)
326         storage: StorageConfig = Field(default_factory=StorageConfig)
327
328         # Allow extra keys during transition so existing config.json files do not break.
329         model_config = {
330             "extra": "allow",
331             "validate_assignment": True,
332         }
333
334         # Temporary migration shim for legacy dict-style access (config.get("section",
{})).get(...)).
335         # Will be removed once all call sites (validators, segmenters, etc.) move to typed
336         # attribute access / the typed config. A nested *section* is returned as a plain
dict
337         # via model_dump, so legacy chained access keeps working ? returning the Pydantic
338         # section model instead breaks it (a model has no .get), which bit every validator
on
339         # real runs. Scalar leaves are returned as-is.
340         def get(self, key: str, default: Any = None) -> Any:
341             _MISSING = object()
342             val: Any = _MISSING
343             if key in type(self).model_fields or hasattr(self, key):
344                 val = getattr(self, key, _MISSING)
345             if val is _MISSING:
346                 # Legacy convenience: find a leaf key inside any nested section. Sections
are
347                 # discovered dynamically from the model fields, so a future top-level
section
348                 # (e.g. `audio`, `jobs`) is covered automatically without editing a

```

```

hardcoded list.
349         for fname in type(self).model_fields:
350             section = getattr(self, fname, None)
351             if isinstance(section, BaseModel) and hasattr(section, key):
352                 val = getattr(section, key)
353                 break
354         if val is _MISSING:
355             return default
356         if isinstance(val, BaseModel):
357             return val.model_dump(mode="json")
358         return val
359
360     def __getitem__(self, key: str) -> Any:
361         val = self.get(key)
362         if val is None and key not in type(self).model_fields:
363             # Re-raise like a real dict for unknown top-level
364             raise KeyError(key)
365         return val
366
367
368     # Convenience type alias for callers that want to annotate
369     Config = AppConfig
370

```

10. assistant (2026-06-20T09:03:21.397Z)

I've confirmed `config.get("indexing", {})` works on the typed config (it has a `.get` shim returning the section as a dict via `model_dump`). So line 183 of `_resolve_train_detector` does indeed call `seg._resolve_runner(...)` with no try/except.

Now let me verify the `wasb_hybrid` and `fusion_hybrid` `_build_native_runner` overrides to confirm which families have native support, and that fusion with tracknet substrate would raise.

11. user (2026-06-20T09:03:22.012Z)

```

1     """WASB-based hybrid play segmenter.
2
3     Phase 2 uses **WASB** (nttcom/WASB-SBDT, MIT) shuttle-trajectory detection via
4     WSL2 (backend/pipeline/detectors/wasb_runner.py). WASB ships pretrained
5     badminton weights and runs today, so this is the live shuttle-motion substrate.
6
7     For the per-video *calibration* goal we mainly use Phase 2 (trajectory ->
8     candidate windows); the windowing thresholds (velocity_thresh / merge_gap /
9     min_window_duration) are what get tuned against golden rally labels.
10
11     The pipeline engine (healthcheck -> trajectory -> windows -> AI handoff -> DB)
12     lives in trajectory_hybrid.TrajectoryHybridSegmenter; this class only wires the
13     WASB runner and identity.
14     """
15     from typing import Any, Dict, Tuple
16
17     from backend.pipeline.segmenters.base import segmenter_registry
18     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
19     from backend.pipeline.detectors.base import DetectorRunner
20     from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
21
22
23     class WasbHybridSegmenter(TrajectoryHybridSegmenter):
24         family = "wasb"
25         display_label = "WASB"
26
27         @property
28         def name(self) -> str:
29             return "wasb_hybrid"
30

```

```

31         @property
32         def description(self) -> str:
33             return ("Two-stage pipeline: WASB shuttle-trajectory detection (WSL2, pretrained
badminton "
34                     "weights) for precise candidate rallies + AI provider for semantic
boundaries.")
35
36         def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
37             cfg = WasbConfig.from_indexing_cfg(idx_cfg)
38             return WasbRunner(cfg), {"weights_path": cfg.weights_path}
39
40         def _build_native_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner,
Dict[str, Any]]:
41             # Lazy import: the native runner pulls no heavy deps, but keep parity with the
stub
42             # seam (imported only when detector_impl='native' is actually selected).
43             from backend.pipeline.detectors.native_wasb_runner import NativeWasbRunner,
NativeWasbConfig
44             cfg = NativeWasbConfig.from_indexing_cfg(idx_cfg)
45             return NativeWasbRunner(cfg), {"weights_path": cfg.weights_path,
46                                           "detector_impl": "native", "device": cfg.device}
47
48
49         segmenter_registry.register(WasbHybridSegmenter)
50

```

12. assistant (2026-06-20T09:03:25.058Z)

Now let me check fusion_hybrid and tracknet_hybrid to confirm the tracknet substrate would raise NotImplementedError, and verify there's no tracknet_hybrid override of `\_build\_native\_runner`.

13. user (2026-06-20T09:03:25.591Z)

```

1         """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
2
3         Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
4         TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5         do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6         shuttle-seeded window via two overridable seams the engine exposes:
7
8         - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9         GPU-free, computed from the trajectory we already have); and, only when the person cue
10        is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11        (``fusion_features``), persisting them into ``candidate_segments.stats`` for the learned
12        fusion (P3). The person path lazily builds ONE `PersonDetector` and only then loads
the
13        torchvision/supervision model ? so the default (person-OFF) path never loads a
detector
14        model nor touches the GPU. (torch/cv2 themselves are imported by the registry
regardless,
15        via the pre-existing yolo_hybrid ? person_detector chain; the lazy part is the model.)
16        - ``_select_for_handoff`` ? optional served Gate A/B: when
``indexing.fusion.min_crossings``
17        (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
are
18        dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
19        **Persisted candidates remain the full superset** regardless, so the offline
experiment
20        harness can re-score any variant.
21
22        Defaults reproduce the substrate exactly: net_crossings/person features are persisted
but
23        nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
seeds

```

```

24     and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
default
25     segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
through the
26     experiment harness (EXPERIMENT_HARNESS.md), never silently.
27     """
28     from typing import Any, Dict, List, Optional, Tuple
29
30     from backend.pipeline.segmenters.base import segmenter_registry
31     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
32     from backend.pipeline.detectors.base import DetectorRunner
33     from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
34     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
35     from backend.pipeline.detectors import rally_gate
36     from backend.pipeline.detectors import fusion_features as ff
37
38
39     class FusionSegmenter(TrajectoryHybridSegmenter):
40         """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
B)."""
41
42         family = "fusion"
43         display_label = "Fusion"
44
45         def __init__(self, db: Any, config: Any):
46             super().__init__(db, config)
47             # Built lazily only if the person cue is enabled (no detector model loaded
otherwise).
48             self._person: Optional[Any] = None
49             # Cache the play-axis/net estimate per trajectory so rally_gate doesn't re-
estimate it
50             # over the whole trajectory once per candidate window (it depends only on
`points`).
51             self._axis_cache_key: Optional[int] = None
52             self._axis_cache: Optional[tuple] = None
53
54             @property
55             def name(self) -> str:
56                 return "fusion_hybrid"
57
58             @property
59             def description(self) -> str:
60                 return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
rallies, "
61                         "net-crossing Gate A + optional person-occupancy Gate B adjudicate them,
and the "
62                         "AI provider sets semantic boundaries.")
63
64             def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
65                 substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
66                 if substrate == "tracknet":
67                     cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
68                     return TrackNetRunner(cfg), {
69                         "weights_path": cfg.weights_path,
70                         "queue_length": cfg.queue_length,
71                         "fusion_substrate": "tracknet",
72                     }
73                 wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
74                 return WasbRunner(wcfg), {"weights_path": wcfg.weights_path, "fusion_substrate":
"wasb"}
75
76             def _build_native_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner,
Dict[str, Any]]:
77                 # Native (no-WSL) path supports the WASB substrate only ? there is no native

```

```

TrackNet
78         # runner yet. Fusion's default substrate is WASB, so this covers the common
case.
79         substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
80         if substrate != "wasb":
81             raise NotImplementedError(
82                 "detector_impl='native' supports the WASB substrate only ? set "
83                 "indexing.fusion.substrate='wasb' (no native TrackNet runner yet).")
84         from backend.pipeline.detectors.native_wasb_runner import NativeWasbRunner,
NativeWasbConfig
85         cfg = NativeWasbConfig.from_indexing_cfg(idx_cfg)
86         return NativeWasbRunner(cfg), {"weights_path": cfg.weights_path,
"detector_impl": "native",
87                                     "fusion_substrate": "wasb", "device": cfg.device}
88
89     def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
90                                frame_width: float, video_path: str,
91                                idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
92         stats = super()._enrich_candidate_stats(cw, points, fps, frame_width,
video_path, idx_cfg)
93         # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
persisted
94         # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
estimated
95         # over the whole trajectory by rally_gate (camera-agnostic); reused across
windows.
96         gated = rally_gate.gate_windows(points, [[cw["start"], cw["end"]]], fps,
97                                               min_crossings=0,
estimate=self._axis_estimate(points))
98         stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0
99
100        fcfg = idx_cfg.get("fusion", {})
101        if fcfg.get("cue_flags", {}).get("person", False):
102            stats.update(self._person_features(cw, points, fps, frame_width, video_path,
fcfg))
103        return stats
104
105        # P3 (learned fusion) first-step landing note (per MULTI_SIGNAL_FUSION_PLAN +
NEXT_STEPS 2026-06-14):
106        # Person features (from _person_features when cue_flags.person) are now persisted
for the harness
107        # `compare` LOVO litmus on the 6 golden videos. Eager-person-compute optimisation
(compute only
108        # for shuttle-seeded windows + padding, not whole video) is already in place via the
lazy
109        # self._person + windowed detections_per_frame. Next: actual harness run + lift
measurement
110        # (owner/hardware) before any served promotion. Flag-gated (default person OFF). See
111        # docs/NEXT_STEPS.md "Rally-detection quality track" and EXPERIMENT_HARNESS.md for
discipline.
112
113    def _axis_estimate(self, points: List[Any]) -> tuple:
114        """Cache rally_gate's (axis, net, span) play-axis estimate per trajectory so
it's
115        computed once per video, not once per candidate window (it depends only on
points)."""
116        key = id(points)
117        if self._axis_cache_key != key or self._axis_cache is None:
118            self._axis_cache_key = key
119            self._axis_cache = rally_gate.estimate_play_axis(points)
120        est = self._axis_cache
121        assert est is not None
122        return est
123
124    def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,

```

```

125         frame_width: float, video_path: str,
126         fcfg: Dict[str, Any]) -> Dict[str, float]:
127     """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
128     frame range and compute the scale/translation-invariant person features. Lazily
builds
129     ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
130     # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
131     from backend.pipeline.detectors.person_detector import PersonDetector
132     if self._person is None:
133         self._person = PersonDetector(self.config)
134
135     frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
136     # round (not truncate): int() would map e.g. 1.999s*30 to frame 59 not 60 ? a
137     # window-edge off-by-one that drops the boundary frame from the shuttle
alignment.
138     start_f, end_f = round(cw["start"] * fps), round(cw["end"] * fps)
139     det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
140     fw = det["frame_width"] or frame_width
141     fh = det["frame_height"]
142     # If we can't trust the frame dims, every normalised feature would silently
collapse
143     # to ~0 ? return an explicit all-zero person-feature row instead of a half-baked
one.
144     if fw <= 0 or fh <= 0:
145         return {k: 0.0 for k in ff.PERSON_FEATURE_NAMES}
146
147     poly = fcfg.get("court_polygon")
148     polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
149     net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
150     shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
151     return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
152                                     court_polygon=polygon, net=net)
153
154     def _select_for_handoff(self, windows: List[Dict[str, Any]],
155                             window_stats: List[Dict[str, Any]],
156                             idx_cfg: Dict[str, Any]) -> List[int]:
157     """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
158     (0) keep every window ? identical to the substrate. Persisted candidates are
untouched (full superset).
159
160     NEVER-EMPTY safeguard: a video that had ?1 window must keep ?1 after gating. If
the
161     thresholds would drop EVERY window (e.g. a short, noisy or one-sided-feed
trajectory
162     where no segment clears ``min_crossings``), the gate falls back to the full
ungated
163     set rather than emit a video with zero rallies. This makes Gate A/B strictly
164     *pruning* ? it can shrink a candidate set but never zero it out ? so layering it
on
165     the substrate is "do no harm": the bounded-windowing / largest-internal-gap
split
166     below it always wins on an otherwise-empty video. Observed on real footage
167     (mahadevpura-1): a single mega-window split into pieces, none clearing the
crossings
168
169     gate ? without this guard the whole clip yielded 0 rallies (no reel)."""
170     fcfg = idx_cfg.get("fusion", {})
171     min_crossings = int(fcfg.get("min_crossings", 0))
172     min_players = int(fcfg.get("min_players", 0))
173     person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
174
175     keep: List[int] = []

```


Reproduced exactly as the finding claims. The `NotImplementedError` propagates uncaught out of `\_resolve\_train\_detector`. Now let me confirm the full `cmd\_train` path actually reaches this line. Note: `cmd\_train` line 219-222 requires `>= 2` labeled videos before reaching `\_resolve\_train\_detector` at line 229. Let me verify the full end-to-end via `wmain.main(["train", ...])` to confirm it crashes rather than returning a clean rc.

19. user (2026-06-20T09:03:57.676Z)

[worker] need >=2 labeled videos for leave-one-video-out; found 0 under /tmp/wtc/corpus/labels/
CLEAN RETURN, rc = 2

20. assistant (2026-06-20T09:04:00.528Z)

The label discovery didn't find the CSVs. The `corpus` is a bare local path. Let me check how `storage.list\_uris` handles a bare path on Windows/this setup ? the issue may be path normalization. Let me debug.

21. user (2026-06-20T09:04:02.428Z)

```
Exit code 1
Traceback (most recent call last):
  File "<string>", line 8, in <module>
    print('listdir:', os.listdir('/tmp/wtc/corpus/labels/'))
                        ~~~~~^~~~~~
FileNotFoundError: [WinError 3] The system cannot find the path specified:
'/tmp/wtc/corpus/labels/'
list_uris result: []
exists: False
```

22. assistant (2026-06-20T09:04:05.321Z)

The Python interpreter is Windows-native (the Bash tool's `/tmp` maps differently than Python's view of the path). The directory creation went to the Git Bash `/tmp` but Python (Windows) can't see it. Let me use a Windows-accessible temp path.

23. user (2026-06-20T09:04:17.143Z)

```
Traceback (most recent call last):
  File "<string>", line 16, in <module>
    rc = wmain.main(['train', '--corpus-uri', os.path.join(base, 'corpus'), '--out-
uri', os.path.join(base, 'out'),
        '--version', '0.0.1', '--config', cfgp, '--
scratch', os.path.join(base, 'scratch'),
        '--trajectory-source', 'detector', '--segmenter', 'tracknet_hybrid'])
  File "C:\Users\avidu\Projects\badminton-highlight-indexer\backend\worker\__main__.py", line
342, in main
    return args.func(args)
           ~~~~~^~~~~~
  File "C:\Users\avidu\Projects\badminton-highlight-indexer\backend\worker\__main__.py", line
229, in cmd_train
    runner = _resolve_train_detector(args, config)
  File "C:\Users\avidu\Projects\badminton-highlight-indexer\backend\worker\__main__.py", line
183, in _resolve_train_detector
    runner, _ = seg._resolve_runner(config.get("indexing", {}))
                ~~~~~^~~~~~
  File "C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\trajectory_hybrid.py", line 85, in _resolve_runner
    return self._build_native_runner(idx_cfg)
           ~~~~~^~~~~~
  File "C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\trajectory_hybrid.py", line 60, in _build_native_runner
    raise NotImplementedError(
...<2 lines>...
    f"Linux-native runner. Use detector_impl='auto' (WSL) or 'stub'.")
```


NotImplementedError: detector_impl='native' is not supported for the 'TrackNet' family ? only WASB has a Linux-native runner. Use detector_impl='auto' (WSL) or 'stub'.
=== UNCAUGHT NotImplementedError -> WORKER CRASH ===

24. assistant (2026-06-20T09:04:25.190Z)

Fully reproduced end-to-end via the public ``main()`` entrypoint. With ``RALLY_DETECTOR_IMPL=native`` and ``--segmenter tracknet_hybrid``, a ``python -m backend.worker train --trajectory-source detector`` invocation crashes with a raw ``NotImplementedError`` traceback that propagates all the way out of ``main()`` ? instead of the function's established graceful pattern (print clear message, return None, which ``cmd_train`` turns into ``return 2``).

Now let me verify the second claim about ``cmd_infer`` exposure. The finding states ``cmd_infer -> process_video -> _resolve_runner`` has the same exposure. Let me check ``process_video`` in ``trajectory_hybrid.py`` (lines 137-372). It calls ``self._resolve_runner(idx_cfg)`` at line 193 with no try/except. And ``cmd_infer`` line 127 calls ``process_video`` inside a ``try/finally`` (not try/except). Let me reproduce the infer path too.

25. user (2026-06-20T09:04:36.149Z)

```
[mov,mp4,m4a,3gp,3g2,mj2 @ 0000019541739480] moov atom not found
C:\Users\avidu\AppData\Local\Temp\wti_c0klmg0l\scratch\in.mp4: Invalid data found when
processing input
Could not detect video duration for
C:\Users\avidu\AppData\Local\Temp\wti_c0klmg0l\scratch\in.mp4: Command '['ffprobe', '-v',
'error', '-show_entries', 'format=duration', '-of', 'default=noprint_wrappers=1:nokey=1',
'C:\Users\avidu\AppData\Local\Temp\wti_c0klmg0l\scratch\in.mp4']' returned non-zero exit
status 1.
Invalid video duration.
[worker] pulled C:\Users\avidu\AppData\Local\Temp\wti_c0klmg0l\in.mp4 ->
C:\Users\avidu\AppData\Local\Temp\wti_c0klmg0l\scratch\in.mp4
[worker] infer done: 0 segment(s); pushed 2 artifact(s):
  C:\Users\avidu\AppData\Local\Temp\wti_c0klmg0l\out\outputs\vidX\intervals.csv
  C:\Users\avidu\AppData\Local\Temp\wti_c0klmg0l\out\outputs\vidX\report.json
CLEAN RETURN, rc = 0
```

26. assistant (2026-06-20T09:04:41.171Z)

Important nuance for the ``cmd_infer`` path. With a fake (invalid) video, ``process_video`` returns early at ``video_duration <= 0`` (line 184-186) **before** reaching ``_resolve_runner`` (line 193). So with a real video the order matters. Let me trace the order in ``process_video``: the ``_resolve_runner`` call is at line 193, which comes **AFTER** the duration check (line 183-186) and **AFTER** ``_probe_fps_width`` (line 187). With a valid video that passes validation, it **WOULD** reach line 193 and crash.

Let me confirm by directly calling ``process_video`` with the duration/fps probing mocked so the real ``_resolve_runner`` is exercised ? this isolates whether the infer path truly has the exposure.

27. user (2026-06-20T09:04:50.556Z)

GEMINI_API_KEY environment variable is not set! To run the gemini handoff, set your API key. In PowerShell: `$env:GEMINI_API_KEY="your_api_key_here"`
NO CRASH (returned)

28. assistant (2026-06-20T09:04:53.792Z)

The default config requires ``GEMINI_API_KEY`` (the AI handoff path), which returns early at line 168-176 **before** ``_resolve_runner`` (line 193). Let me set ``skip_ai_handoff=true`` so it bypasses the API key check and reaches ``_resolve_runner``.

29. user (2026-06-20T09:05:01.941Z)

Traceback (most recent call last):

catches it.

The finding is accurate, the line reference (`backend/worker/\_\_main\_\_.py:183`) is precise, both the train and infer exposures reproduce, and the proposed fix is correct and idiomatic for this codebase.

31. user (2026-06-20T09:05:33.385Z)

Structured output provided successfully

32. assistant (2026-06-20T09:05:41.617Z)

I've verified the finding and submitted my structured verdict.

****Verdict: real**** (severity corrected to ****minor****)

The finding is accurate and I reproduced it end-to-end through the public entrypoint. With `RALLY\_DETECTOR\_IMPL=native` and a non-WASB trajectory segmenter (`--segmenter tracknet\_hybrid`, or `fusion\_hybrid` with `substrate=tracknet`), `\_resolve\_train\_detector` at `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\worker\\_\_main\_\_.py:183` calls `seg.\_resolve\_runner(...)` with no try/except, so the `NotImplementedError` from the base `\_build\_native\_runner` propagates uncaught out of `cmd\_train` ? `main` ? crash with a raw traceback ? instead of the function's established "print + return None ? rc=2" pattern that every other failure branch uses. The `cmd\_infer` ? `process\_video` ? `\_resolve\_runner` exposure is also real (process_video:193, only wrapped in try/finally).

I downgraded blocker/major to ****minor**** because it only fires on an explicit misconfiguration (the new native flag + a non-WASB segmenter; the default `--segmenter wasb\_hybrid` and default infer paths are unaffected), with no data/correctness impact ? the operation correctly refuses, just ungracefully. The proposed fix (wrap in `try/except NotImplementedError: print + return None`, mirroring the sibling branches) is correct and idiomatic.